State-Based Conformance Testing for Agent Tool Protocols

OpenBot

***Abstract*—Agent tool protocols carry calls across transport, access, and application boundaries. A valid reply shape does not show that a server keeps the right state after a token change, a repeated write, or a resumed request. This design and analysis paper proposes a reusable test method for these boundaries. It joins a small state model, generated request traces, explicit rejection rules, and read-back checks. Dependency records keep generated and reduced request sequences meaningful. Coverage counts state transitions and boundaries. Comparisons use a shared contract. A fully defined synthetic fixture supplies accounts, grants, note effects, and one-use continuations. We give conditional oracle and reduction arguments, and executable finite experiments. Exhaustive traces of at most four note operations detect all five defined mutants. Adding a final read exposes more traces of hidden writes, but both checks detect every mutant in this bounded set. The full network generator and cross-implementation study remain untested. The contribution is a precise joint contract and its checkable limits, not a new general conformance theory.**

***Index Terms*—Model-based testing, Protocol conformance, Stateful testing, Agent tools**

I. Introduction

A tool protocol can accept well-formed requests and still break its contract. A server can hide a write tool from a read-only caller yet accept a direct call to that tool. It can return an error for an old revision after it has already deleted a record. A client can close a stream but keep using an access token after its grant has ended. Each defect depends on a sequence. A test that treats each message as a separate case can miss the cause.

Conformance means that observed behavior stays within the allowed behaviors. A schema defines message fields. A state model defines allowed outputs and changes after earlier messages. An agent tool can ask for input and receive it later under a different account. The model must represent that binding across requests.

This paper proposes state-based conformance testing for agent tool protocols. It uses the Model Context Protocol, or MCP, as the wire setting and defines a small synthetic application contract. Endpoint discovery supplies the target; the research variable is state-based defect finding.

The current protocol revision matters. MCP `2026-07-28` removes the connection initialization handshake and protocol sessions. Requests carry version and capability metadata. Cross-call application state still exists. The official change list also adds `server/discover` and changes follow-up input into multi round-trip requests. Thus a state test must not invent a required handshake for a stateless protocol. [1] Our model places state in grants, workspaces, continuations, and outstanding requests.

The method joins access rights, effects, and follow-up input in one model. A generator reaches useful states before making a targeted invalid move. Reply checks and state checks support comparison under a shared profile. A reducer preserves the objects and access history that make a failure meaningful.

This is a design, analysis, and finite-model study. Its artifacts are a synthetic contract, a proposed network generator, executable note and one-use-handle models, defined faulty baselines, and conditional arguments. The completed experiments test small models; they do not run an MCP server. We separate these results from the planned study across independent implementations.

II. Synthetic Fixture and Operation Alphabet

The fixture defined here is a proposed test application, not a deployed service. It has accounts `a` and `b`, protected resources `r0` and `r1`, workspace names `w0` and `w1`, and write keys `k0`, `k1`, and `k2`. A text is a string of at most 16 letters from `a` and `b`. Both accounts may obtain either read-only scope or read/write scope for either resource. Each grant has a fresh family identifier. Two grants for one account are separate families.

Time is an integer tick. Reset sets time to zero and clears all mutable state. An authorization code lasts 10 ticks. A grant family lasts 60 ticks from issue. An access token expires at the earlier of 20 ticks after issue or the family deadline. A refresh token expires at the family deadline. A continuation lasts 20 ticks. A workspace lasts 100 ticks from its first successful write; later calls do not extend it. An item is expired when current time equals or exceeds its deadline. These numbers are fixture definitions, not measured timing values.

There is one registered public client `c` and one exact callback address `u`. The adapter assigns `u` a loopback test address. Use two PKCE verifiers: 43 repetitions of `a`, or of `b`, with their S256 challenges. `authorize` takes an account, resource, scope set, challenge, and allow-or-deny choice. Denial returns `Denied` without a code. Allow creates a code bound to all given values, `c`, and `u`. `exchange` takes the code, client, callback, resource, and verifier. It succeeds only while the code is unused, unexpired, and all bindings match. A failed check preserves the code. Success atomically consumes it and creates a grant family, access token, and refresh token. Exchange failures return `GrantRejected`.

TABLE I
THE COMPLETE ABSTRACT OPERATION ALPHABET. THE TEXT DEFINES THE GUARDS, REPLIES, AND STATE CHANGES.

Operation	Required effect
`authorize`, `exchange`	Create bound code; consume it once to issue tokens.
`refresh`, `revoke`	Rotate a refresh token; end one grant family.
`list`, `read`	Return visible tools or the current workspace.
`write(w,k,text)`	Create one note or replay its saved receipt.
`delete(w,id,rev)`	Delete only at the given current revision.
`begin-input`, `resume-input`	Create, validate, and consume one continuation.
`advance`, `drop-response`, `reset`	Control time, observation loss, and initial state.

`refresh` takes a refresh token and a scope subset. It requires a live family and an unused, unexpired token bound to `c` and the same resource. Success atomically consumes that token and issues a new pair with no added scope. Invalid refresh state returns `GrantRejected`; extra scope returns `ScopeRejected`. Earlier access tokens remain valid until expiry or family revocation. `revoke` takes any issued token handle and marks its family revoked. Repeating revocation or naming an unknown token succeeds without change. The fixture makes revocation effective before replying.

Workspaces are keyed by resource, account, and workspace name. A missing workspace reads as revision zero with no notes. `write` checks its key before capacity. A matching key and text return the exact saved receipt without a write. A matching key with other text returns `KeyConflict`. Otherwise, if two notes already exist it returns `Full`. A new write adds one note, raises the revision by one, uses that revision as note ID, and saves the resulting view as its receipt. `delete` returns `RevisionConflict` when its expected revision differs. At the current revision, deleting a missing note returns the unchanged view. Deleting a present note removes it and raises the revision by one. It preserves all write receipts. Workspace expiry removes notes, revision, and receipts together. The capacity guard is reachable: write with `k0`, then `k1`, then `k2` in one workspace. The first two calls create two notes at revision two; the third returns `Full` and leaves that view unchanged.

Two tools, `ask-label` and `ask-code`, support follow-up input. `begin-input` takes a tool and text and returns a fresh handle plus one requested answer named `answer`. A valid answer is either letter `a` or `b`. `resume-input` must repeat the account, resource, tool, and text, and provide exactly that answer key. It checks every binding and the deadline before atomically consuming the handle. Any invalid attempt preserves a live handle. Completion returns the tool, original text, and answer. An invalid or repeated completion returns `InputRejected`. These calls never write notes.

All application operations first check access. Missing, wrong-resource, expired, or revoked access tokens yield HTTP 401. A live token lacking a required scope yields 403. Reads, lists, and input tools need read scope; writes and deletes need write scope. `list` hides tools that need absent scope. Tool conflicts return a tool-error result with the named error class. Successful `begin-input` returns an input-required result; other successful tool calls return complete results. Invalid typed fields return a protocol input error. Rejected calls do not change application state. This fixture defines these rules explicitly; only the applicable wire rules are claims about MCP.

III. FOUNDATIONS AND RELATED WORK

Model-based testing starts from a description of allowed behavior. Tretmans presents testing with labelled transition systems and the input-output conformance relation, ioco. In that setting, outputs after a specification trace must be allowed by the specification. The theory also treats quiescence, the absence of output, as observable. [2] This gives our oracle a clear starting point. We use a bounded, timed engineering profile; we do not claim that a finite run proves ioco for an unbounded network service.

AFLNet grows sequences from recorded network exchanges and uses response codes as state feedback. It can explore a server without a full protocol grammar. [3] Our method instead requires a small semantic model. This cost buys distinctions that response codes alone can hide. Two requests may both return HTTP 200 while one repeats a receipt and the other repeats a write. Greybox coverage, when available, can supplement the model but cannot supply that semantic verdict.

RESTler infers request dependencies from an API description and uses response feedback to build useful stateful sequences. [4] We adopt the need to preserve producer-consumer links. A created token or continuation is a value produced by an earlier step and used later. Our proposed addition is a joint model of rights, one-use input state, and effects, with explicit negative transitions and read-back witnesses. This is a focused combination for agent tool protocols, not a claim to invent stateful API testing.

Delta debugging reduces a failure-inducing input by testing smaller candidates. It must distinguish a passing candidate from one that cannot be meaningfully tested. [5] We carry this distinction into trace reduction. Deleting the token-creation step should not count as fixing an access defect. It makes the candidate invalid unless the trace has another valid producer for that token.

Model-based shrinking also predates this design. Koopman, Achten, and Plasmeijer use a model to search for shorter black-box failure traces. [6] Recent model-centric shrinking separates faults tied to a model state from faults tied to history that the state may omit. [7] A short path to the same visible notes can lose a receipt or grant history. We retain those histories and test the same failed rule after reduction. This is a contract-specific choice within model-based reduction, not a new reduction principle.

Timed conformance needs a precise meaning for absent output. Schmaltz and Tretmans show the role of a finite output bound in timed conformance relations. [8] A quiet network interval does not by itself establish quiescence. Our

profile must state its output bound, clock controls, and request completion barrier. Without the required controls, a timeout leaves the verdict open.

Standards supply only part of the oracle. OAuth resource indicators bind a requested token to a protected resource. [9] PKCE binds an authorization code exchange to proof held by the client. [10] Token revocation defines a way to end token use, with related-token behavior set by the server and applicable rules. [11] MCP defines transport access and the follow-up request pattern. [12], [13] The synthetic fixture adds note, revision, grant-family, and one-use continuation rules. Reports must identify which layer a failed rule belongs to. A fixture rule cannot silently become a claim about every MCP server.

IV. State Model and Assumptions

A test profile is a versioned contract for one target class. It declares mandatory features, optional features, input classes, time bounds, reset support, and permitted outputs. The initial profile uses the defined fixture over MCP `2026-07-28` Streamable HTTP. Native tools need separate application adapters and contracts.

The full model state is a tuple `S = (V, A, G, T, W, K, Q, P, C)`. `V` is the chosen protocol version and declared capability set. `A` maps code handles to account, client, redirect address, proof challenge, resource, scopes, expiry, and use status. `G` maps grant families to status, account, and deadline. `T` maps token handles to scopes, resource, family, type, expiry, and use status. `W` maps resource, account, and workspace triples to deadline, revision, and notes. `K` adds a write key to that triple and stores original input and receipt. Its entries end with that workspace. `Q` maps continuation handles to their binding and status. `P` holds pending requests and open streams. `C` is controlled time.

A handle is a symbolic name for a server-issued value. The trace stores `access-1`, not a raw token. The adapter binds it to a fresh value on replay while preserving relationships, such as shared grant families. A protected wire log retains exact bytes for diagnosis.

This is an extended state machine: transitions can read values as well as a small control state. A finite abstraction groups values by their role. Examples include valid or expired token, same or other account, matching or changed input, current or stale revision, and fresh or used continuation. The abstraction is for search and coverage. Exact revision and note values remain in the oracle. Grouping all nonempty workspaces together must not erase a wrong revision.

`advance(n)` adds a nonnegative number of ticks. Expiry is applied before the next access or state check. `drop-response` cuts a selected response after request delivery without undoing server work. A separate harness barrier must confirm request termination before a later read can settle its effect. The barrier reveals no result. Without it, the request remains pending and the verdict is inconclusive. `reset` clears the target, adapter, and model together. The initial state has no grants, codes, tokens, workspaces, receipts, continuations, or pending requests. Its version and declared capability set are fixed.

The profile assumes isolation and no unmodelled writer. A fresh workspace is not a reset because old tokens may remain live. Targets without complete reset need a fresh isolated instance; otherwise reset-dependent tests are inconclusive.

For each input, the transition function applies expiry, checks access, checks typed fields and operation guards, then atomically commits the defined effect. It returns the named error class or success value. Failure leaves live business state unchanged. Raw error prose is not compared. An access failure may still remove an expired temporary record; it must not create a note or consume a live continuation.

Invalid inputs have the defined rejection transitions. Inputs outside those rules yield an inconclusive verdict, rather than an invented contract violation.

V. Generating Useful Traces

The generator chooses an uncovered obligation before it chooses bytes. It visits obligation classes in round-robin order, chooses a shortest reachable prefix within the class, and uses the saved seed to break ties. Breadth-first search explores prefixes of at most eight operations in alphabet order. Each prefix gets at most two fresh-reset attempts. Unreached obligations remain reported, so one hard obligation cannot take the whole budget. An obligation might be "repeat a write key after a deletion" or "resume under another account after a valid continuation was issued." It fills symbolic handles from earlier outputs and chooses literal values from the finite fixture domains. Negative fields add one wrong-type value, one unknown handle or binding, and text of length 17. Time choices are zero, one, and distances to the nearest deadline minus one, at the deadline, or plus one. Negative distances are omitted. Reports retain the seed, profile version, and obligation.

Positive steps build reachability. Negative steps cross one guard at a time. For instance, a code exchange first uses the wrong proof while keeping the client, redirect address, and resource correct. A later step uses the right proof. This sequence checks both rejection and whether a failed proof consumed a usable code. Testing several bad fields at once would leave the responsible rule unclear. Multi-fault cases are a later campaign with separate coverage.

Algorithm 1: Reach, perturb, observe

1) Choose an uncovered profile obligation.
2) Find a model prefix that reaches its guard.
3) Reset the target; bind fresh symbolic values.
4) Run the prefix and check each observation.
5) Change one selected input or state boundary.
6) Run the target step and required read-back.
7) Save new coverage or a failure witness.
8) Reduce failures; replay from a clean reset.

Fig. 1. Proposed online generation loop. Prefix failure stops the trace; the runner never credits an obligation it did not reach.

The search queue stores abstract state signatures with a short replay prefix. Prefixes are not snapshots of a live server.

Replaying them after reset avoids hidden dependence on a long campaign. Retain more than one prefix when they differ in a relevant history, such as whether a key has a stored receipt. The signature retains the full symbolic state, with handles renamed by creation order. It cannot merge states with different receipt histories.

After a valid prefix, the runner selects one of four step classes: normal operation, boundary input, cross-identity substitution, or transport disturbance. Boundary values include just before and just after expiry, the same and next revision, empty and maximum allowed text, and a reused key with changed input. Time moves use a test clock where the adapter supports one. A remote clock test brackets the boundary with known uncertainty and avoids giving a hard verdict inside that interval.

Transport disturbance is kept separate from business rejection. Dropping a reply after a write may leave the model with two possible states: unchanged or changed once. The runner must read the workspace or retry with the same application key to resolve the uncertainty. It cannot assume that a closed HTTP stream rolled back the write. The current MCP revision removes stream redelivery and requires a new request identifier for a reissued request; the application key can still remain stable. [1]

Concurrency is a bounded extension. The first campaign uses one outstanding operation. A second campaign issues two operations against the same code, refresh token, or continuation behind a start barrier. It accepts only outcomes allowed by a sequential ordering under the one-use contract of that object. The profile does not require a particular winner. This targets consumption races without claiming full concurrent coverage.

VI. Oracles and Cross-Implementation Checks

An oracle decides whether an observation is allowed. We propose three levels. The wire oracle checks encoding, required metadata, response correlation, and declared schemas. The transition oracle checks the allowed output and updates a set of possible states. The invariant oracle uses later observations to check a fact that must remain true across steps, such as account isolation or absence of a write after access rejection.

Let `B` be the set of model states consistent with the trace so far. For input `a` and normalized observation `o`, the next set contains every model successor allowed by `a` and `o` from a state in `B`. An empty successor set is a failure. A larger set expresses uncertainty. This set-valued update matters after response loss, allowed timing variation, or unordered notifications. Choosing one convenient state would create both false failures and missed defects.

Read-back checks make side effects visible. After a rejected delete, read the workspace under a still-valid account and compare revision and note content. After a rejected write, read unchanged state, then use a fresh permitted key to show that the target remains usable. After a wrong-account continuation attempt, let the original account resume it. A preserved continuation shows that an invalid caller did not consume another caller's input state. These checks are witness steps, not cleanup.

Silence has a limited meaning. A requested response that never arrives can be a liveness failure only under an agreed response bound and a healthy transport. Otherwise it is an unresolved run. A quiet subscription is often valid. An absent update notice does not by itself prove that a write did not occur. The runner records timeouts, stream close, and no output as different observations.

Differential testing sends the same symbolic trace to several implementations. Normalization renames opaque handles consistently and removes harmless field order. It preserves identity relationships, schema fields, error classes, revisions, effect counts, and revision-based note IDs. Adapters map one abstract command into each target's wire form. Adapter tests must show that these mappings preserve the chosen profile.

A difference needs review. A model-forbidden response is a violation. Different permitted responses are expected. Missing rules require contract review. Several SDKs can share one bug, so agreement cannot prove conformance. Note results are comparable only under the shared note contract.

VII. Worked Failure and Reduction

Define a faulty baseline with every fixture rule unchanged except successful deletion. After removing a present note and raising the revision, it removes every receipt whose saved view contains that note. This exact rule wrongly treats a receipt as part of current note storage. The baseline is hypothetical. Start with a valid read/write token, an empty workspace, and time zero. No time advances during the following trace.

TABLE II
Illustrative receipt-loss trace. Access setup is a recorded dependency before step 1. The repeated receipt and current state have different revisions by design.

Step	Input	Required observation
1	Write text `a`, key `k0`	Receipt at revision 1, note 1
2	Delete note 1, expected 1	Current view: revision 2, empty
3	Write text `a`, key `k0` again	Original receipt at revision 1
4	Read workspace	Current view: revision 2, empty

A correct step 3 returns the old receipt without a new note. The read-back still sees revision 2. The faulty server instead treats the key as new. It returns revision 3 with a new note, and step 4 confirms that the note exists. The revision calculation is explicit: the first write adds one, the deletion adds one, and a forbidden second write adds one. This gives `0 + 1 + 1 + 1 = 3` instead of the required current revision `0 + 1 + 1 = 2`.

A reply-only check can find the wrong receipt at step 3. The read-back adds evidence of a changed effect, rather than a wrongly serialized response alone. Define a second faulty baseline that retains receipts but, on matching-key replay, increments the revision, appends a new note, leaves receipts unchanged, and returns the old receipt. Step 3 passes a reply

check for that baseline. Step 4 alone exposes its forbidden write. The pair thus separates output conformance from effect conformance.

The same structure supports a negative access trace: issue a read/write token, write once, revoke its grant family, try a new write with the old token, then read through a separate valid grant for the same account. Rejection alone is insufficient; the last read must show no new note. The family-wide rule is the fixture's declared contract. A general OAuth test must use the provider's stated revocation scope and propagation bound.

Reduction starts with the failing invariant identifier and its witness step. It removes unrelated page reads and tool calls, then tries blocks of operations. A dependency graph links each use to the step that produced its token, note, key receipt, or continuation. A candidate must retain a valid path for these links. Removing step 1 from the table would remove both the note and the receipt needed to expose the defect.

After structural reduction, value reduction makes text and names shorter, moves revisions toward the nearest failing boundary, and reduces delays while preserving their order relative to expiry. Replays must reproduce the same invariant failure. A new failure, such as an invalid token caused by broken setup, does not qualify. The final output includes setup, trigger, witness, expected state, observed state, and rejected reduction attempts that explain a required step.

VIII. Analytical Properties and Coverage

The oracle has a conditional soundness property. Assume the model includes every behavior allowed by the profile, the adapter observes faithfully, reset reaches the declared initial state, and time uncertainty is represented. Start with the correct initial belief set. At each step, a conforming target's actual state has an allowed successor matching its observation. Induction keeps at least one consistent state in the belief set. The oracle therefore cannot declare failure through an empty set. The assumptions, especially model completeness, are essential. This is a local argument for the proposed monitor, not a proof about the entire implementation.

There is also an observation limit. Let two targets have the same allowed observation traces under every input sequence available to an adapter. Any adaptive test using only those inputs and observations has the same set of possible verdicts on both targets. To see this, couple the test's random choices. The empty histories agree. Equal histories give the same next input, and the targets permit the same next observations. Induction preserves this agreement. Thus no such test can distinguish a hidden effect that changes none of its observations. A final read enlarges the observation interface; it does not make every outside effect visible.

Control is a separate limit. The generator can choose an input and a reset prefix. It cannot choose an allowed nondeterministic response unless the adapter provides that control. A coverage denominator may include an edge that the target never takes within the budget. That edge remains uncovered; a retry limit cannot turn it into a successful test. Each adapter must declare four capabilities: reset, clock control, request completion, and effect observation. Missing capabilities make the affected obligations inconclusive.

For reduction, define the measure as trace length followed by total input size. Each accepted reduction strictly decreases this lexicographic measure and keeps the same reproduced failure. The process terminates for a finite trace and finite candidate list. If it exhausts every allowed single-step removal at the end, the result is one-step minimal under those removals. It need not be globally shortest. Dependency constraints and nondeterministic replay can both prevent further reduction.

Coverage has explicit denominators. Transition coverage is the number of exercised reachable abstract edges divided by the reachable edges in the frozen bounded profile. Guard coverage counts each declared guard with both a passing and a failing witness. Boundary coverage counts tested value classes, including expiry and revision neighbors. Rights coverage counts applicable triples of account relation, token state, and operation class. Ordered-pair coverage counts adjacent abstract transitions, such as delete followed by receipt replay.

These metrics stay separate. A single percentage would hide an untouched access boundary behind many ordinary reads. As an illustrative calculation, suppose a toy profile has 12 reachable transitions and a run visits 9. Its transition coverage is `9 / 12 = 75%`. If the same run checks only 2 of 8 rights obligations, rights coverage is `2 / 8 = 25%`. These numbers describe a toy model; they are not experimental measurements.

An edge earns coverage only when its precondition is established and its relevant witness completes. A timed-out prefix earns no coverage for the planned final guard. Model-unreachable obligations need a recorded reachability argument. Target-unreachable obligations, such as optional features it does not offer, appear as inapplicable for that profile. Failures to reach mandatory features remain visible. Code coverage, when collected, is reported separately from contract coverage.

Bounded reachability search costs time proportional to the explored graph plus outgoing edges when each abstract state is visited once. Concrete replay adds the length of each selected prefix and its network cost. The product of accounts, scopes, resource states, and continuation states can grow quickly. Partial profiles and targeted pair obligations control this cost; they do not remove the risk of missed longer histories.

IX. Completed Finite Experiments

The supplied artifact `papers_11_12.py` runs two kinds of checks. A deterministic campaign enumerates all words of length zero through four over seven note operations. The operations are writes `(k0,a)`, `(k0,b)`, `(k1,b)`, and `(k2,a)`, deletes of note 1 at expected revision 1 or 0, and read. There are `1+7+49+343+2401 = 2,801` traces and 10,738 operation steps per target. Each trace starts empty and receives one extra final state observation. These are direct model calls, not HTTP requests.

An event-fold oracle stores accepted add and remove events, and stores receipts apart from current notes. It derives expected revisions from the event count. The target uses mutable

note state. Every correct-target trace agrees with the oracle. A metamorphic check renames all keys and swaps the two text letters, then checks that observations change only by that swap. This checks a required symmetry; it does not establish model independence.

Five mutants each remove one rule: keep receipts after delete; replay without an effect; reject changed input for a used key; enforce capacity two; or reject a stale revision. The table gives traces exposing each defect. The reply check includes any reads already present in the trace. The combined check adds one final read. Its extra observation has a cost, so this comparison is an oracle ablation, not an equal-request search comparison.

TABLE III
COMPLETED BOUNDED MUTATION STUDY. EACH ROW HAS 2,801 TRACES; COLUMNS COUNT TRACES, NOT INDEPENDENT DEFECTS.

Removed rule	**Reply**	**Final state**	**Either**
Keep deleted-note receipt	120	120	120
Replay has no write	320	892	952
Reject changed key input	468	0	468
Keep capacity two	240	240	240
Reject stale revision	1,476	796	1,476

The final read adds 632 exposing traces for the hidden-write mutant. Both strategies still detect all five mutants in the full bounded set. This result supports the need for an effect witness on some traces; it does not show a higher distinct-defect yield, faster search, or better coverage of real servers. No confidence interval is needed for this complete finite enumeration. The generalization unit is the fixture and its stated mutations, not the number of traces.

A dependency-aware reducer starts with six operations: write, read, delete, read, replay, and final read. It keeps the last read as the witness. It requires the first write as the producer of the deleted note and replayed receipt. Nine candidate replays reduce the trace to write, delete, replay, and read. All single removals then either break a dependency or lose the same state failure. This shows one-step minimality for this case, not global optimality or broad reduction performance.

A separate campaign runs 100 note commands for each seed 0 through 29. It checks exact state and saved receipts after each step. One-use handles are checked just before, at, and after deadlines 10, 20, and 60, with a wrong binding before a valid attempt. The total is 13,537 assertions. Assertion count is a reproducibility check, not a measure of research strength. These experiments cover the note and handle submodels. They omit OAuth exchange, network faults, real concurrent consumption, and the full proposed state search.

X. Proposed Evaluation

The evaluation would answer three questions. Does explicit state improve confirmed defect discovery over fixed cases and unguided sequences? Do access-aware and read-back checks find distinct defect classes? Does dependency-aware reduction yield smaller reproducible traces within a fixed replay budget?

Use at least three independently built protocol stacks with the same pinned protocol revision. Select servers or clients from three language ecosystems whose released versions support the profile. Verify support before enrollment. Run both directions: generated clients against servers, and scripted servers against clients. Do not count several wrappers around one SDK as independent implementations. Record shared SDK and authorization libraries as dependence between targets.

Each target receives the same note-tool contract for application tests. Native tools form a separate generalization set with their own reviewed adapters. Freeze all profiles and mutations before the main runs. Mutations deliberately add one plausible defect: remove an account key, lose a receipt on delete, skip a scope check, accept a used refresh token, consume a continuation before binding checks, or accept a stale revision. Use only isolated local targets. Independent review determines which mutations change observable contract behavior; equivalent mutations are excluded with reasons.

Baselines are a fixed suite, schema-valid random requests, dependency-valid random sequences, and a state-feedback sequence fuzzer. The fixed suite runs each successful operation once from fresh setup, each named rejection once, and the four-step receipt trace. Random baselines use the same literal domains. Each receives the same starting credentials, reset interface, transport access, and request budget. Run a separate time-budget comparison because reset and replay costs differ. Report adapter and model authoring effort as a cost of the method.

For each target and strategy, propose 30 independent seeds and a fixed budget of 10,000 requests per seed. These are study settings, not completed trials. Pair seeds and target versions where meaningful. Reserve part of each budget for replay and reduction so a method cannot hide its diagnosis cost. The main endpoint is confirmed distinct defects found. Secondary measures are time and requests to first finding, the coverage families, false reports, unresolved runs, and final trace length.

Use paired bootstrap intervals over seed-level differences within each target. Report target-level effect sizes before any aggregate. Seeds are not independent implementations, and requests are not independent samples. Time-to-finding includes unsuccessful runs as censored observations. Broad superiority needs evidence across the target set.

The reduction study must also compare generic trace deletion and a model-path reducer. It counts candidate replays, setup failures, and reproductions of the same failed rule. A smaller trace with a new failure does not count as a successful reduction.

Ablations remove one part at a time: negative guards, cross-account substitution, read-back witnesses, ordered-pair search, or dependency-aware reduction. Keep budgets equal. A separate experiment replaces the model oracle with differential agreement to measure errors from shared behavior. Reviewers classify every report using the pinned contract and a minimal replay. Preserve rejected reports as well as confirmed ones so the precision estimate is auditable.

XI. Limits and Reproducibility

A model can repeat the same mistake as its implementation. The fixture rules here are the contract; no reference server defines correctness by agreement alone. A second reviewer should derive expected traces directly from these rules and check every deliberate departure from public standards. Models for native tools need the same independent contract review.

Finite abstractions can merge states with different futures. Long histories, storage failures, and weakly ordered replicas may escape the initial model. The one-use concurrency test does not establish general linearizability. A timeout outside a controlled environment is not proof of protocol failure. Token revocation in a remote deployment may have a declared delay that the local fixture does not have.

The note service is deliberately small. It exposes clean read-back state and has no external business effect. Real tools may send mail or start jobs whose effects cannot be safely reversed or directly read. Such tools need isolated test tenants and explicit effect witnesses. If these are absent, the method can check wire behavior and some access rules but must leave effect conformance unresolved.

Reproduction requires executable target artifacts, dependency versions, profile and adapter versions, reset procedure, seeds, symbolic traces, normalized and protected raw logs, and clock settings. Record operating system, transport proxy, target versions, and resource limits. The original and reduced trace must both replay. Public reports use symbolic credentials and synthetic note text. They identify the exact violated rule and its source.

A proposed implementation has four separate modules: a pure model, a network runner, wire adapters, and a reducer. The runner owns request cancellation and ensures reset completion before replay. Each trace records setup and cleanup failures separately from protocol verdicts. A successful capability-list call establishes only that observation; it cannot count as evidence of write, access-change, or continuation conformance.

XII. Conclusion

Agent tool conformance depends on history even when the transport is stateless. Grants, repeated writes, revisions, and follow-up input create observable state boundaries. This paper gives a concrete method to reach those boundaries, perturb one rule, check both replies and effects, and reduce a failure without losing its cause. The completed finite study makes the effect-witness distinction and its limits concrete. All defined mutants are observable within the chosen bound, and additional final reads reveal hidden writes on more traces. The proposed network evaluation must still show whether the joint contract earns its modelling cost across independent implementations.

References

[1] Model Context Protocol, "Key Changes: Specification 2026-07-28." Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/changelog.mdx

[2] J. Tretmans, "Model Based Testing with Labelled Transition Systems," in *Formal Methods and Testing*, Springer, 2008, pp. 1–38. doi: 10.1007/978-3-540-78917-8_1.

[3] V.-T. Pham, M. Böhme, and A. Roychoudhury, "AFLNet: A Greybox Fuzzer for Network Protocols," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification*, 2020, pp. 460–465. doi: 10.1109/ICST46399.2020.00062.

[4] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API Fuzzing," in *Proceedings of the 41st International Conference on Software Engineering*, 2019. [Online]. Available: https://www.microsoft.com/en-us/research/publication/restler-stateful-rest-api-fuzzing/

[5] A. Zeller and R. Hildebrandt, "Simplifying and Isolating Failure-Inducing Input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002, doi: 10.1109/32.988498.

[6] P. Koopman, P. Achten, and R. Plasmeijer, "Model-Based Shrinking for State-Based Testing," in *Trends in Functional Programming*, in Lecture Notes in Computer Science, vol. 8322. Springer, 2014, pp. 107–124. doi: 10.1007/978-3-642-45340-3_7.

[7] S. van den Broek, A. Oprescu, and T. C. Ruys, "Bug Localisation in Model-Based Testing using Model-Centric Shrinking," in *Proceedings of the 2nd ACM International Workshop on Future Debugging Techniques*, Association for Computing Machinery, 2024, pp. 9–15. doi: 10.1145/3678720.3685315.

[8] J. Schmaltz and J. Tretmans, "On Conformance Testing for Timed Systems," in *Formal Modeling and Analysis of Timed Systems*, in Lecture Notes in Computer Science, vol. 5215. Springer, 2008, pp. 250–264. doi: 10.1007/978-3-540-85778-5_18.

[9] B. Campbell, J. Bradley, and H. Tschofenig, "Resource Indicators for OAuth 2.0," technical report RFC 8707, 2020. doi: 10.17487/RFC8707.

[10] N. Sakimura, J. Bradley, and N. Agarwal, "Proof Key for Code Exchange by OAuth Public Clients," technical report RFC 7636, 2015. doi: 10.17487/RFC7636.

[11] T. Lodderstedt, S. Dronia, and M. Scurtescu, "OAuth 2.0 Token Revocation," technical report RFC 7009, 2013. doi: 10.17487/RFC7009.

[12] Model Context Protocol, "Authorization: Specification 2026-07-28." Accessed: Oct. 27, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization/index

[13] Model Context Protocol, "Multi Round-Trip Requests: Specification 2026-07-28." Accessed: Oct. 27, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/basic/patterns/mrtr